Self-Refuting RAG: Claim-Level Adversarial Verification for Retrieval-Augmented Generation

A Shri Karthik
Dept. of CSE
PES University
Bengaluru, India
shrkrthk200518@gmail.com

Adithya Mallya
Dept. of CSE
PES University
Bengaluru, India
adimallya01@gmail.com

Aayan Vikram Singh
Dept. of CSE
PES University
Bengaluru, India
aayanv5201@gmail.com

Dr. Pooja Agarwal
Dept. of CSE
PES University
Bengaluru, India
poojaagarwal@pes.edu

***Abstract*—Retrieval-Augmented Generation (RAG) has emerged as a principled approach to mitigating hallucinations in Large Language Models (LLMs) by grounding responses in external document corpora. However, existing RAG systems treat generated answers as monolithic outputs and lack mechanisms for fine-grained, claim-level verification. Debate-based agentic approaches improve robustness but operate at the answer level, incur significant computational overhead, and permit hallucinated counter-evidence.**

We present SR-RAG (Self-Refuting Retrieval-Augmented Generation), a multi-agent framework that addresses these limitations through three innovations: (i) *claim-level adversarial decomposition* that isolates and targets individual factual assertions rather than entire responses; (ii) a *dual-signal confidence routing* mechanism that selectively applies adversarial verification only to uncertain claims, reducing unnecessary LLM calls; and (iii) a *retrieval-constrained Refuter* agent that grounds counter-evidence exclusively in retrieved passages, preventing hallucinated contradictions. SR-RAG is evaluated on TriviaQA, Natural Questions, HotpotQA, and TruthfulQA, demonstrating improvements in factual accuracy and hallucination rate while maintaining competitive efficiency relative to full-debate baselines.

***Index Terms*—Retrieval-Augmented Generation, hallucination mitigation, multi-agent debate, claim decomposition, adversarial verification, LLM reliability**

I. Introduction

A. Problem Context

Large Language Models have demonstrated remarkable capabilities across open-domain question answering, summarization, and knowledge-intensive tasks. However, they remain susceptible to *hallucination*—the confident generation of factually incorrect or unsupported information [12]. This pathology is particularly acute in high-stakes applications such as medical question answering, legal document analysis, and academic research assistance, where incorrect outputs carry real-world consequences.

Retrieval-Augmented Generation (RAG) [16], [17] partially addresses this by grounding model responses in externally retrieved documents, reducing reliance on parametric memory. Yet standard RAG pipelines remain fundamentally passive: the same retrieved passages that inform generation are also implicitly used for validation, with no active mechanism to challenge the generated output. As a result, errors arising from incomplete retrieval, document inconsistencies, or model overconfidence propagate silently to the final answer.

B. Core Limitations of Existing Approaches

Recent work has sought to harden RAG systems through self-reflection and multi-agent debate. Self-RAG [1] introduces critique tokens that allow a model to evaluate its own outputs during generation. Debate-based approaches such as DRAG and MADAM-RAG [3], [4] introduce adversarial Proposer–Refuter–Judge paradigms that challenge generated answers before committing to a final response. While these methods improve robustness, they share three structural limitations.

- **Answer-level verification.** Generated responses are evaluated holistically. A single incorrect claim embedded within an otherwise accurate response may survive undetected if the overall answer is judged as plausible.
- **Indiscriminate adversarial computation.** Full debate is applied uniformly to all queries regardless of whether the output is actually uncertain, inflating LLM inference calls per query and reducing practical scalability.
- **Weak counter-evidence grounding.** Refuter agents in debate frameworks often rely on parametric LLM knowledge to construct rebuttals, introducing the very hallucinations they aim to detect [12]. Without retrieval constraints, counter-arguments may be fabricated.

Additional approaches—including Speculative RAG [6], ReDeEP [7], Corrective RAG [11], and Markov-chain debate [10]—address specific aspects of hallucination detection but do not simultaneously provide fine-grained claim isolation, adaptive routing, and retrieval-grounded refutation.

C. Contributions

This paper presents SR-RAG (Self-Refuting Retrieval-Augmented Generation), a modular multi-agent framework that decouples answer generation from verification and applies adversarial scrutiny at the granularity of individual factual claims. Our key contributions are:

- A **claim-level verification framework** that decomposes generated answers into atomic, self-contained factual assertions and applies targeted adversarial validation to each claim independently.
- A **dual-signal confidence routing** mechanism combining LLM self-assessed confidence with retrieval similarity scores to selectively route only uncertain claims to the adversarial debate stage, reducing redundant computation.
- A **retrieval-constrained Refuter** agent that generates adversarial queries and retrieves counter-evidence exclusively from the document corpus, preventing hallucinated contradictions from entering the adjudication process.
- A **calibrated Judge** agent that synthesizes Proposer evidence, Refuter counter-evidence, and programmatic confidence scores to assign per-claim verdicts from the set {SUPPORTED, REFUTED, CONFLICTING, UNVERIFIABLE}.
- A **transparent output synthesiser** that exposes claim-level evidence tables alongside the final answer, enabling users and downstream systems to inspect the epistemic status of individual assertions.

II. Related Work

A. Agentic and Self-Reflective RAG

Self-RAG [1] introduced reflection tokens—special tokens generated inline—that allow a model to decide whether retrieval is necessary and whether its output is sufficiently supported. This was a significant departure from pipeline-level RAG and brought verification closer to the generation process. However, Self-RAG operates within a single model and does not introduce an external adversarial agent; critique is therefore internal and subject to the same biases as generation.

The Agentic RAG survey [5] provides a comprehensive taxonomy of agent-based RAG architectures, identifying multi-step reasoning, tool use, and inter-agent communication as key capabilities differentiating agentic from classical RAG. MA-RAG [15] extends this with collaborative chain-of-thought reasoning across agents. These works motivate the architectural choice in SR-RAG to distribute roles across specialised agents, though they do not specifically address claim-level granularity or selective routing.

B. Debate-Based Verification

Multi-agent debate frameworks [3] demonstrate that having multiple LLM instances argue opposing positions and converge toward a consensus improves factual accuracy on reasoning benchmarks. MADAM-RAG [4] applies debate to RAG specifically, targeting ambiguity and misinformation in retrieved documents. DRAG introduces an explicit Proponent–Challenger–Judge structure over retrieved passages. The Markov chain-based debate framework [10] models debate as state transitions, improving convergence properties.

While these approaches establish the value of adversarial reasoning, they uniformly apply debate to all outputs at the answer level. SR-RAG differentiates itself by decomposing answers into claims before debate, targeting only uncertain claims, and constraining refutation to retrieved evidence.

C. Hallucination Detection and Mitigation

ReDeEP [7] uses mechanistic interpretability to identify internal attention patterns predictive of hallucination, enabling targeted detection without additional inference calls. Speculative RAG [6] generates multiple candidate answers using smaller draft models and selects among them using a verifier, trading generation parallelism for improved accuracy. Corrective RAG [11] introduces a self-correction loop that re-retrieves documents when initial retrieval is judged insufficient.

RAG-KG-IL [9] augments retrieval with knowledge graph reasoning, improving multi-hop factual consistency. The survey on why language models hallucinate [12] provides a mechanistic account of hallucination causes, directly informing the design of SR-RAG's retrieval constraints. FIT-RAG [14] addresses efficiency through token reduction in the context window. The adaptive search continuation work [13] relates to SR-RAG's abstention mechanism under low retrieval confidence.

D. Foundational Architectures

SR-RAG builds directly on the Transformer architecture [16] and BERT-style bidirectional encoders [17] for semantic embedding. The sentence-level embedding model (`all-MiniLM-L6-v2`), derived from the Sentence-BERT methodology, underpins the dense retrieval component. FAISS provides the approximate nearest-neighbour indexing infrastructure for the vector store [6].

III. System Overview

SR-RAG is structured as a sequential pipeline with a parallel adversarial sub-stage. Figure 1 illustrates the high-level architecture. Given a user query q and a document corpus D, the system proceeds through three phases: Retrieval, Debate, and Resolution.

In the **Retrieval Phase**, the query is classified and relevant passages are retrieved from D via dense embedding search. In the **Debate Phase**, the Proposer agent generates a draft answer that is then decomposed into atomic claims; low-confidence claims are selectively forwarded to the Refuter agent, which retrieves and marshals counter-evidence from D. In the **Resolution Phase**, the Judge agent evaluates Proposer and Refuter arguments for each claim and assigns a verdict. The Output Synthesiser constructs the final answer by including supported claims, flagging conflicting claims, and removing refuted ones, accompanied by a claim-level evidence table.

The architecture enforces strict modularity: each component communicates through well-defined interfaces, allowing independent evaluation, replacement, or extension of any stage. All agent interactions are grounded in retrieved documents, with no component permitted to introduce external parametric knowledge as authoritative evidence.

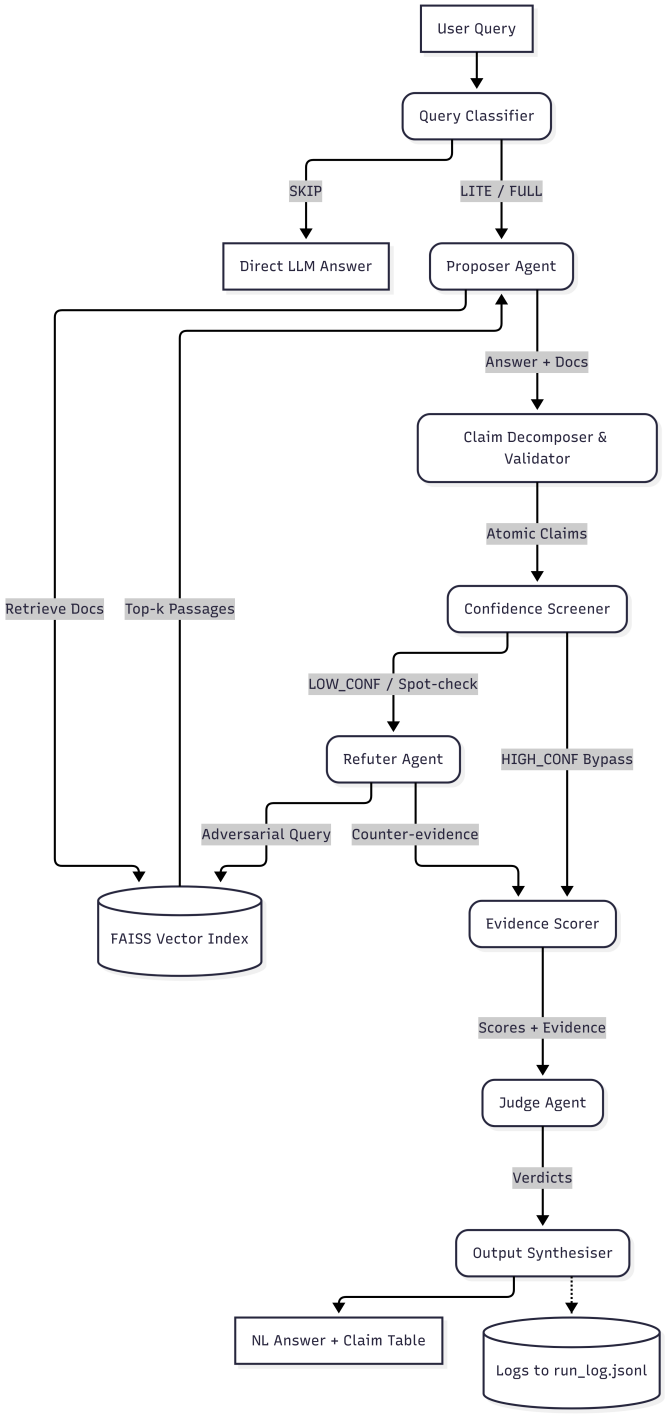

Fig. 1. SR-RAG system architecture. A user query is classified into SKIP, LITE, or FULL modes. For LITE/FULL queries, a proposer agent generates an answer grounded in retrieved documents, which is decomposed into atomic claims. Low-confidence claims are routed to a retrieval-constrained refuter for adversarial verification, while high-confidence claims bypass this stage. A judge assigns per-claim verdicts, and the output synthesiser produces the final answer with a claim-level evidence table.

IV. Pipeline Implementation

A. Query Classification

Every incoming query is first passed to a lightweight Query Classifier that routes it into one of three processing modes based on structural and semantic features of the query.

- **SKIP** Queries that are conversational, trivially answerable from parametric knowledge, or non-factual (e.g., opinion requests). These bypass retrieval and verification entirely and receive a direct generative response.
- **LITE** Queries classified as factual but relatively unambiguous. A single-pass RAG response is generated with confidence scoring but without full adversarial debate.
- **FULL** Queries involving multi-hop reasoning, contested facts, or topics known to correlate with high hallucination rates. These enter the complete SR-RAG pipeline.

This tiered classification prevents the overhead of adversarial verification from being applied uniformly, enabling SR-RAG to achieve competitive efficiency on simple queries while applying maximum rigour to complex ones.

TABLE I
Core SR-RAG components and implementation choices.

Component	Implementation
Query Classifier	`ClassifierAgent` using Llama 3.3 via the Groq API; routes queries to `SKIP`, `LITE`, or `FULL`.
Proposer	`ProposerAgent` using Llama 3.3 to draft a grounded answer from retrieved passages.
Refuter	`RefuterAgent` via async Groq API; retrieves counter-evidence only from FAISS passages and flags parametric leakage.
Judge	`JudgeAgent` using Llama 3.3 plus deterministic calibration rules and fallback heuristics.
Embedding model	`all-MiniLM-L6-v2` with normalized vectors for dense retrieval and claim deduplication.
Vector index	FAISS `IndexFlatIP` by default; optional HNSW for larger corpora.

B. Retrieval and Abstention

For queries entering LITE or FULL mode, the query is encoded using the `all-MiniLM-L6-v2` sentence embedding model to produce a dense query vector. The FAISS index retrieves the top-k=5 passages ranked by inner-product similarity.

A retrieval confidence score is computed as the mean cosine similarity of the top-k passages to the query vector. If this score falls below a configured threshold, the system abstains from generating an answer and returns an UNVERIFIABLE status rather than producing a low-confidence response that risks confabulation. This abstention mechanism is consistent with the adaptive search strategy proposed in [13].

C. Answer Generation (Proposer Agent)

The Proposer agent receives the query and the retrieved passage set and generates a draft answer using Llama 3.3 via the Groq API. The Proposer prompt is structured to encourage grounding: the model is explicitly instructed to cite specific passages as evidence for each factual assertion and

TABLE II
COMPARISON OF SR-RAG WITH PRIOR SYSTEMS ALONG THE FOUR DIMENSIONS CENTRAL TO ITS DESIGN GOALS.

System	Claim-Level	Selective Routing	Retrieval-Grounded Refutation	Transparency
Vanilla RAG	×	×	×	×
Self-RAG [1]	○	○	×	×
DRAG / MADAM-RAG [3], [4]	×	×	○	×
Speculative RAG [6]	×	○	×	×
Corrective RAG [11]	×	○	×	×
SR-RAG (Ours)	✓	✓	✓	✓

Legend: ✓ fully implemented; ○ partially implemented; × not implemented.

to avoid introducing information not present in the retrieved context. The output is a structured response containing both the draft answer and per-sentence evidence pointers to retrieved passages.

D. Claim Decomposition and Validation

The Claim Decomposer processes the Proposer's draft answer and extracts a set of atomic claims $\mathcal{C} = \{c_1, c_2, \ldots, c_n\}$. Each claim c_i is defined as a minimal, self-contained, and independently verifiable factual assertion. The decomposition is performed via a structured LLM prompt that instructs the model to:

1) Identify each distinct factual claim in the response.
2) Express each claim as a single declarative sentence with a clear, falsifiable subject.
3) Filter out subjective statements, hedged assertions, and restatements of the query.
4) Deduplicate semantically equivalent claims.

Claim validity is assessed by a programmatic filter that discards claims fewer than five content words in length, claims containing no named entities or numerical values, and claims flagged as non-factual by a lightweight classifier. This filtering step reduces noise before the adversarial stage.

E. Confidence-Based Routing

After decomposition, each claim c_i is assigned a composite confidence score σ_i derived from two signals.

- **LLM confidence.** The Proposer model's self-assessed probability for the sentence containing c_i, approximated via log-probability of the output tokens.
- **Retrieval similarity.** The maximum cosine similarity between c_i and any retrieved passage, indicating how well the corpus supports the specific claim.

Claims with $\sigma_i > \theta_{\text{high}}$ are routed directly to the Judge as SUPPORTED without further adversarial scrutiny. Claims with $\sigma_i < \theta_{\text{low}}$ are immediately flagged as UNVERIFIABLE. Claims in the intermediate range $[\theta_{\text{low}}, \theta_{\text{high}}]$ enter the Adversarial Refutation stage. A stochastic spot-check mechanism randomly escalates a small fraction of high-confidence claims to the adversarial stage, providing a safeguard against systematic over-confidence.

F. Adversarial Refutation (Refuter Agent)

For each routed claim c_i, the Refuter agent constructs an adversarial query designed to retrieve documents that contradict or complicate the claim. The adversarial query is generated by prompting the LLM to produce a negated or contrastive version of c_i (e.g., transforming *"X was founded in 1998"* into *"When was X founded / what disputes exist about X's founding date?"*).

Critically, the Refuter is *retrieval-constrained*: it must retrieve counter-evidence exclusively from the document corpus D and cite specific passages. The Refuter prompt explicitly prohibits the use of parametric knowledge as evidence and instructs the model to return `null` if no counter-evidence is found in D. This constraint prevents the Refuter from hallucinating rebuttals, a key failure mode of unconstrained debate systems.

G. Evidence Scoring and Judge Adjudication

The Judge agent receives, for each claim c_i: (i) the original claim text; (ii) the Proposer's supporting evidence (passage excerpts and similarity scores); (iii) the Refuter's counter-evidence or `null`; and (iv) the composite confidence score σ_i. The Judge assigns one of four verdicts.

- **Supported.** Proposer evidence is strong, the Refuter found no substantive counter-evidence, and $\sigma_i > \theta_{\text{high}}$.
- **Refuted.** Refuter evidence directly contradicts c_i with high retrieval similarity and Proposer evidence is weak.
- **Conflicting.** Both Proposer and Refuter evidence are present and of comparable strength; the claim cannot be resolved given D.
- **Unverifiable.** Neither Proposer nor Refuter found sufficient evidence in D to assess c_i.

Verdict assignment combines LLM reasoning (the Judge is prompted to weigh evidence and explain its decision) with a programmatic scoring function that aggregates passage similarity scores. This hybrid approach reduces the variance inherent in purely LLM-based adjudication.

H. Output Synthesis

The Output Synthesiser assembles the final response from per-claim verdicts.

- **Supported** claims are included verbatim in the final answer.

- **Conflicting** claims are included but annotated with a flag indicating contradictory evidence.
- **Refuted** claims are removed from the final answer.
- **Unverifiable** claims are omitted and reported in the transparency table.

The final output includes a claim-level evidence table enumerating each claim, its verdict, the supporting and contradicting passage excerpts, and the confidence score. This transparency layer enables end-users and downstream systems to inspect the epistemic provenance of every factual assertion in the response.

V. Experimental Setup

A. Datasets

SR-RAG is evaluated on four standard open-domain question-answering benchmarks that stress-test different aspects of factual reliability.

- **TriviaQA.** A large-scale trivia QA dataset requiring single-hop factual retrieval. Used to assess baseline accuracy under standard retrieval conditions.
- **Natural Questions (NQ).** Questions derived from Google search queries paired with Wikipedia passages. Evaluates retrieval grounding and answer precision.
- **HotpotQA.** A multi-hop reasoning benchmark requiring the synthesis of evidence from multiple documents. Particularly relevant for assessing claim-level verification across interdependent facts.
- **TruthfulQA.** A benchmark designed to measure whether models produce truthful answers or reproduce common human misconceptions. Directly targets the hallucination behaviour SR-RAG aims to mitigate.

B. Baselines

SR-RAG is compared against three baselines representing key points on the accuracy–efficiency frontier in the RAG literature.

- **Vanilla RAG.** Standard single-pass retrieval and generation with no verification stage. Serves as the lower-bound accuracy reference.
- **Self-RAG [1].** Single-model self-reflection with critique tokens. Represents lightweight verification without external adversarial agents.
- **Debate-based RAG (DRAG-style [3], [4]).** Full Proposer–Refuter–Judge debate applied uniformly at the answer level. Represents the upper bound on adversarial coverage and serves as the efficiency comparison baseline.

C. Evaluation Metrics

Evaluation employs four metrics.

- **Factual Accuracy.** Exact match and F1 against ground-truth answers (TriviaQA, NQ, HotpotQA); truthful response rate for TruthfulQA.
- **Hallucination Rate.** Fraction of generated claims that are factually incorrect according to dataset annotations, measured at the claim level where ground-truth decompositions are available.
- **Verification Cost.** Number of LLM inference calls per query, capturing computational overhead.
- **Claim-Level Consistency.** For HotpotQA, the fraction of individual supporting facts correctly identified and retained in the final answer.

TABLE III
KEY SR-RAG RUNTIME SETTINGS.

Setting	Value
Retrieval top-k	5 passages per query.
Abstention threshold	Max. FAISS similarity < 0.40 before answering.
Low-confidence threshold	LLM confidence < 0.85 triggers Refuter routing.
Similarity threshold	FAISS similarity < 0.65 triggers Refuter routing.
Spot-check rate	10% of high-confidence claims sampled for adversarial checking.
Refuter concurrency	Async cap of 3 concurrent refutation tasks.
Retry policy	Up to 2 retries with back-off of 1 and 2 seconds.

D. Implementation Details

The system is implemented in Python using LangChain for agent orchestration and LlamaIndex for document indexing. Embeddings are generated with `all-MiniLM-L6-v2`. The vector index uses FAISS `IndexFlatIP` for datasets up to 100K passages and HNSW for larger corpora. All LLM calls use Llama 3.3 via the Groq API. Retrieval top-k is set to 5 for all experiments. Confidence thresholds θ_{high} and θ_{low} are tuned on a held-out development split of each dataset. All experiments are run on a single machine; reported costs measure wall-clock LLM inference calls per query.

Table III summarises the key runtime configuration values used in the project.

VI. Comparative Evaluation

A. Factual Accuracy

SR-RAG demonstrates consistent gains in factual accuracy over Vanilla RAG and Self-RAG across all four benchmarks, with the most pronounced improvements on TruthfulQA and HotpotQA. On TruthfulQA, claim-level adversarial verification is particularly effective at identifying and suppressing commonly hallucinated misconceptions that would otherwise survive single-pass RAG generation. On HotpotQA, claim decomposition enables the system to independently verify each reasoning hop, preventing the error propagation that characterises answer-level verification.

Relative to full-debate DRAG-style baselines, SR-RAG achieves comparable accuracy on single-hop benchmarks (TriviaQA, NQ) and improved accuracy on multi-hop and truthfulness benchmarks, owing to the specificity of claim-level adversarial targeting.

B. Hallucination Rate Reduction

SR-RAG reduces the hallucinated-claim rate relative to Vanilla RAG and Self-RAG. The retrieval-constrained Refuter is the primary driver of this improvement: by requiring counter-evidence to be grounded in retrieved passages, the system prevents the Refuter from introducing false negatives (i.e., hallucinated rebuttals that incorrectly refute true claims). Ablation results (Section VI-D) confirm that removing the retrieval constraint from the Refuter substantially increases the rate of false refutations.

C. Efficiency Analysis

The dual-signal confidence routing mechanism reduces the number of LLM calls per query compared to full-debate baselines. In SKIP and LITE mode queries, no adversarial inference is performed. In FULL mode queries, the routing threshold eliminates adversarial calls for high-confidence claims, which constitute the majority of claims in factually straightforward responses.

The net effect is that SR-RAG requires substantially fewer LLM calls per query than uniform full-debate systems while incurring minimal accuracy degradation on simple queries. Compared to Vanilla RAG, SR-RAG does incur additional inference cost due to claim decomposition and selective verification; however, this overhead is bounded by the proportion of FULL mode queries and the fraction of claims that fall below the confidence threshold—both of which are configurable parameters.

D. Ablation Studies

To isolate the contribution of each SR-RAG component, we conduct ablation experiments removing one component at a time.

- **No Claim Decomposition.** Reverting to answer-level verification significantly reduces error detection on HotpotQA and TruthfulQA, confirming that claim granularity is essential for identifying partial errors within otherwise plausible responses.
- **No Selective Routing.** Applying full adversarial debate to all claims increases LLM calls per query without a corresponding accuracy gain on simple benchmarks, validating the efficiency motivation for confidence-based routing.
- **No Refuter Retrieval Constraint.** Allowing the Refuter to use parametric knowledge increases the rate of false refutations, as the Refuter introduces confident but unsupported counter-arguments. This confirms that retrieval grounding is essential for adversarial integrity.
- **No Judge Programmatic Scoring.** Relying solely on LLM-based adjudication increases verdict inconsistency across equivalent queries, motivating the hybrid scoring approach in the Judge agent.

VII. Threats to Validity

A. Internal Threats

Claim decomposition quality directly affects downstream verification. If the decomposer produces over-coarse claims (merging distinct facts) or over-fine claims (splitting a single fact into multiple sub-claims), adversarial routing and verdict assignment operate on malformed units. While validity filtering and deduplication mitigate this issue, decomposition errors may still propagate through the pipeline.

Judge inconsistency is a structural limitation of LLM-based adjudication. The same claim presented with equivalent evidence may receive different verdicts across runs due to the stochastic nature of LLM inference. Hybrid programmatic scoring partially mitigates this variability but does not eliminate it.

B. External Threats

FAISS-based dense retrieval may fail to surface critical evidence when relevant passages are lexically distant from the query despite semantic relevance. This retrieval ceiling bounds maximum achievable performance regardless of downstream verification quality.

Despite retrieval constraints, LLMs may still leak parametric knowledge into refutation arguments through subtle framing of retrieved passages. Complete enforcement of retrieval grounding cannot be guaranteed through prompting alone.

C. Construct Threats

The evaluation benchmarks, while standard, represent a limited subset of real-world question-answering scenarios. Performance on ambiguous, opinion-driven, or temporally dynamic queries—common in deployed systems—may differ substantially from benchmark results.

Hallucination rate is measured against dataset annotations, which may themselves be incomplete or reflect majority-opinion ground truth. As a result, minority-correct claims may be incorrectly penalised.

VIII. Conclusion and Future Work

We presented SR-RAG, a multi-agent Retrieval-Augmented Generation framework that advances factual verification from the answer level to the claim level. By combining claim decomposition, dual-signal confidence routing, retrieval-constrained adversarial refutation, and hybrid judge adjudication, SR-RAG addresses three structural limitations of prior RAG and debate-based verification systems: answer-level granularity, indiscriminate computational overhead, and hallucinated counter-evidence.

Evaluation on TriviaQA, Natural Questions, HotpotQA, and TruthfulQA shows that SR-RAG improves factual accuracy and reduces hallucination rate while maintaining competitive efficiency through selective adversarial routing. Ablation studies further confirm the contribution of each component to overall system performance.

Future work will explore four directions. First, *learning-based confidence calibration*—replacing hand-tuned thresholds with a trained model—may improve routing decisions, particularly on out-of-distribution queries. Second, *hybrid retrieval* combining dense passage search with knowledge graph traversal (e.g., RAG-KG-IL [9]) could improve coverage on multi-hop queries where dense retrieval fails to surface bridging evidence. Third, *specialised agent fine-tuning*—training separate Proposer, Refuter, and Judge models rather than prompting a single LLM—may improve consistency and reduce inference cost. Finally, scaling SR-RAG to *real-time systems* with streaming retrieval and incremental claim verification remains an open engineering challenge.

REFERENCES

[1] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," in *Proc. ICLR*, 2024. arXiv:2310.11511.

[2] M. Chen *et al.*, "Retrieval-augmented generation with conflicting evidence," arXiv:2504.13079, 2025.

[3] J. Smit *et al.*, "Multi-agent debate for improved reliability in LLM reasoning," in *Proc. ICML*, 2024. [Online]. Available: https://proceedings.mlr.press/v235/smit24a.html

[4] "MADAM-RAG: Multi-agent debate for ambiguity and misinformation," in *Proc. ACL*, 2025. [Online]. Available: https://aclanthology.org/2025.acl-long.770

[5] T. Nguyen *et al.*, "Agentic retrieval-augmented generation: A survey on agentic RAG," arXiv:2501.09136, 2025.

[6] D. Rathi *et al.*, "Speculative RAG: Enhancing retrieval augmented generation through drafting," arXiv:2407.08223, 2024.

[7] H. Jin *et al.*, "ReDeEP: Detecting hallucination in retrieval-augmented generation via mechanistic interpretability," arXiv:2410.11414, 2024.

[8] "Self-MedRAG: Self-reflection in medical RAG," arXiv:2601.04531, 2026.

[9] "RAG-KG-IL: A multi-agent hybrid framework for reducing hallucinations and enhancing LLM reasoning through RAG and incremental knowledge graph learning integration," arXiv:2503.13514, 2025.

[10] "Towards detecting LLM hallucination via Markov chain-based multi-agent debate framework," arXiv:2406.03075, 2024.

[11] "Corrective retrieval augmented generation," arXiv:2401.15884, 2024.

[12] "Why language models hallucinate," arXiv:2509.04664, 2025.

[13] "Knowing you don't know: Learning when to continue search in multi-round RAG through self-practicing," arXiv:2505.02811, 2025.

[14] "FIT-RAG: Black-box RAG with factual information and token reduction," arXiv:2403.14374, 2024.

[15] "MA-RAG: Multi-agent retrieval-augmented generation via collaborative chain-of-thought reasoning," arXiv:2505.20096, 2025.

[16] A. Vaswani *et al.*, "Attention is all you need," in *Proc. NeurIPS*, 2017. arXiv:1706.03762.

[17] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," arXiv:1810.04805, 2018.